Tarea 2: Torneo de Komikan Concurrente

Sistemas Operativos – Nicolás González – Noviembre de 2026

1. Diseño del esquema de sincronización

Armé el sistema en tres niveles, cada uno con su propia primitiva POSIX y con una regla simple sobre qué protege.

Primitiva	Protege	Para qué
`sem_t mesas_disponibles` (valor inicial M)	Cupo de mesas activas	Nunca hay más de M mesas. Si no hay cupo, la pareja espera bloqueada.
`lock_matchmaker` + `cond` por pareja	Pareja pendiente de rival	El primer jugador espera en la `cond`; el segundo lo empareja y dirige la partida.
`lock_tablero` (mutex robusto)	Casillas, posición del puma, capturas	Todo movimiento (leer, validar y escribir) ocurre en una sola sección crítica.
`lock_juez` + `cv_juez`	Ganador y causa de término	Las fichas avisan al juez sin que este haga espera activa.
`atomic_bool terminada`	Fin de la partida	Las 13 fichas lo consultan sin tomar ningún lock.

Exclusión mutua en los movimientos. Cada mesa tiene un único mutex que protege todo su tablero. Una ficha toma el candado, mira las casillas que le importan (destino libre; para el puma, también la casilla intermedia y la de aterrizaje del salto), aplica el movimiento y evalúa la victoria, y recién ahí lo suelta. Como comprobar y actuar pasan juntos dentro de la misma sección crítica, si dos fichas apuntan a la misma casilla la segunda ya la ve ocupada y elige otra jugada. No existe el caso "vi la casilla libre pero ya no lo estaba". Un detalle que me costó: cuando el puma captura, el hilo del perro capturado sigue vivo, así que guardo en el tablero qué ficha ocupa cada casilla y marco al perro como capturado. Al despertar, ese hilo lo ve y termina (ver sección 2).

Juez. Cada mesa tiene un hilo juez que inicializa el estado, imprime el tablero cada 500 ms, verifica que el estado cumpla las reglas (un solo puma y $12 -$ capturas perros) y detecta el fin: 6 capturas (gana el Puma), puma sin jugadas ni saltos posibles (ganan los Perros), abandono o SIGINT. Después espera a sus fichas con `pthread_join`, destruye mutexes y `cond`, y como último paso hace `sem_post` para liberar el cupo.

Resiliencia. *Abandono:* cada ficha escribe un latido (una marca de tiempo) en cada ciclo y mantiene una bandera de que sigue viva, que solo borra si termina con normalidad. En cada ciclo de 100 ms el juez revisa las 13 fichas; si alguna sigue marcada como viva pero lleva más de `MAX_DELAY` más 1 s sin latir, el juez detecta la anomalía, declara ganador al oponente por abandono y cierra la mesa. Además, el mutex del tablero es robusto (`PTHREAD_MUTEX_ROBUST`): si el jugador cayó reteniendo el candado, el siguiente hilo que lo pide recibe `EOWNERDEAD`, lo declara consistente y sigue, y el juez, antes de destruir el mutex, lo toma y suelta una vez para limpiarlo. La caída se puede simular con `FALLO <mesa><PUMA|PERROS><ms>` en el `config.txt`: el hilo termina sin avisar a nadie. *SIGINT:* el manejador solo pone `torneo_activo = 0` (es lo único seguro de hacer en un manejador). El `main` deja de aceptar jugadores; los jueces, las fichas y las esperas por semáforo o por pareja usan esperas con timeout de 100 ms y comprueban la bandera. Las mesas terminan en orden, todo se libera y el programa sale. Todos los hilos se crean con SIGINT bloqueado, así que el único que atiende la señal es el principal.

2. Análisis de riesgos de concurrencia

Deadlock. Revisé las cuatro condiciones de Coffman y rompí la de espera circular. Un hilo de ficha toma a lo más dos locks y siempre en el mismo orden (`lock_tablero` → `lock_juez`). El juez nunca

pide el del tablero mientras sostiene el suyo (lo suelta antes de imprimir). `lock_matchmaker` se suelta antes de bloquearse en el semáforo y antes de jugar, de modo que nadie espera un cupo reteniendo ese candado. Como además hay un solo candado por tablero, no existe el escenario clásico de dos fichas que se bloquean casillas cruzadas.

Livelock. No uso `trylock` ni reintentos inmediatos, por lo que no hay hilos que se cedan el paso indefinidamente. Si una ficha no tiene jugada, simplemente duerme su retardo aleatorio (50–300 ms) y vuelve a mirar. Ese jitter también evita que dos fichas choquen siempre al mismo tiempo.

Starvation. Los mutex y semáforos POSIX no garantizan orden FIFO, así que en teoría una ficha o una pareja podría ser adelantada varias veces. En la práctica el riesgo es muy bajo: el candado de una mesa se pide en promedio unas 75 veces por segundo (13 fichas, un retardo medio de 175 ms) y se mantiene unos microsegundos, o sea que casi siempre está libre. Para las parejas, cada partida termina, así que el cupo se libera de forma continua. Para un torneo mucho más grande habría que agregar una cola o ticket explícito.

Otros riesgos que encontré. (i) *Perro fantasma:* al probar, el validador del juez detectó más perros de los que debían existir. El perro capturado seguía moviéndose desde su casilla antigua y "revivía". Se arregló con la marca de captura descrita arriba. (ii) *Despertar perdido:* las `cond` siempre se esperan dentro de un `while` que revisa un predicado bajo lock y con timeout, y las señales se envían con el lock tomado. (iii) *Uso tras liberar:* la mesa la libera solo el juez, y la pareja la libera solo el primer jugador después de que el segundo suelta el lock por última vez.

3. Evaluación

Decisión principal: un candado por tablero. La alternativa era un mutex por casilla, que permite más paralelismo, pero la captura del puma toca tres casillas a la vez (origen, medio y destino) y la victoria depende de todo el tablero. Eso obliga a ordenar los locks o usar `trylock` con reintentos, con riesgo de deadlock y de livelock, y evaluar el "puma acorralado" sería mucho más difícil. El candado único protege una sección que recorre como máximo 28 casillas y mantiene la ocupación del candado en el orden de 10^{-4} (estimación a partir de los tiempos, no una medición), mientras que cada ficha pasa la mayor parte del tiempo durmiendo. La concurrencia real que se pierde es mínima y se gana un diseño mucho más fácil de razonar y de comprobar.

	Un mutex por tablero (elegido)	**Mutex por casilla**	**Sin locks (atómicos)**
Complejidad	Baja	Alta (orden, 3 locks)	Muy alta
Riesgo de deadlock/livelock	Ninguno	Real	Livelock posible
Concurrencia	Suficiente (ver arriba)	Mayor, poco útil aquí	Máxima
Victoria/estado global	Trivial	Requiere tomar todos	Muy difícil

Resultados de las pruebas. Con $N = 24$ y $M = 3$ el máximo de mesas simultáneas observado fue 3 (12 partidas, 0 errores de invariante). Con valgrind (`-leak-check=full`) no hubo fugas ni errores, ni con SIGINT ni con fallos inyectados. ThreadSanitizer, AddressSanitizer y UBSan no reportaron problemas. Se verificó la victoria de los Perros con un puma encerrado, el abandono de ambos bandos y el cierre ordenado con Ctrl+C incluso con llegadas lentas (cortadas al llegar 30 de 100 jugadores).

Limitaciones. Usé hilos y no procesos, así que la caída de un jugador se simula terminando su hilo con el candado tomado y sin avisar, que es justo el caso que cubren la detección por latidos y el mutex robusto. El umbral de latidos (`MAX_DELAY` más 1 s) hace que el abandono se detecte con un pequeño retraso. La semilla fija la secuencia de decisiones y retardos de cada ficha, pero el orden entre hilos lo decide el planificador, así que no se garantiza que dos corridas sean idénticas bit a bit. La estrategia es simple: el puma prefiere capturar y los perros se mueven al azar, por lo que el Puma gana casi siempre. Y las esperas con timeout de 100 ms cuestan algo de CPU, pero lo hacen robusto frente a SIGINT sin depender de funciones que no son seguras en un manejador de señales.